Department of Electrical and Electronics Engineering

Final Project Book

Camera-Based Moving Target Detection, Tracking, and Wireless Reporting System

Students:

Rabea Faraj - [redacted]
Aslan Bi Gush - [redacted]

Industrial Supervisor: None

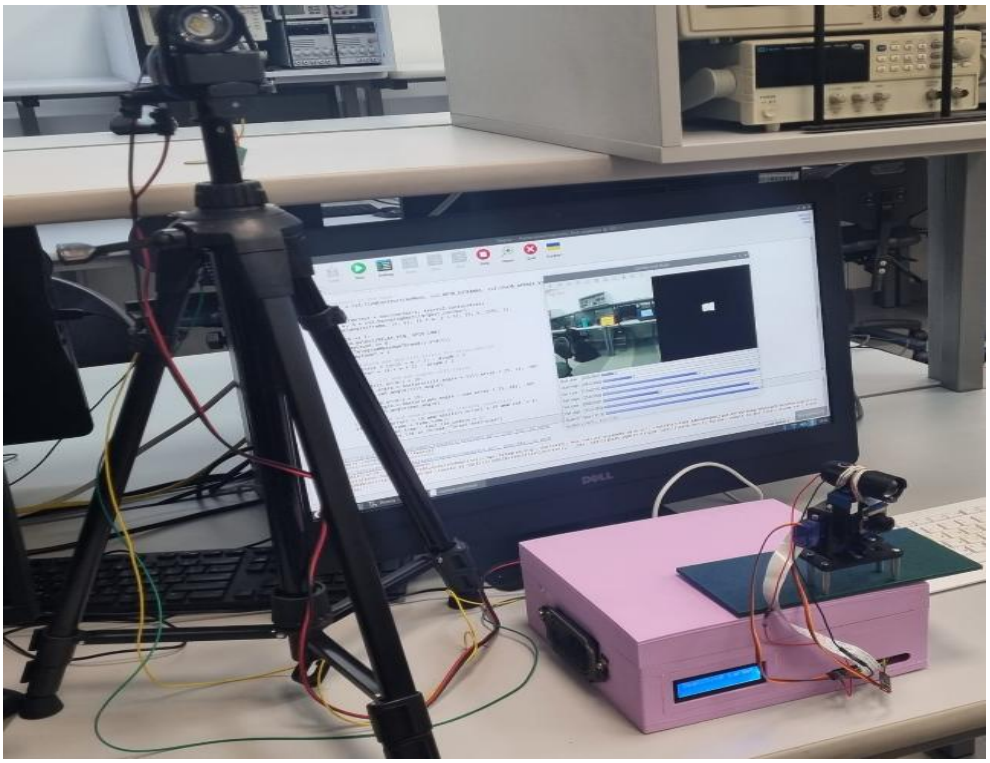
College Supervisor: Shimon Pitelson, [redacted]

Date: 10 November 2024

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science (B.Sc.) in Electrical and Electronics Engineering

This project was carried out as an internal capstone project at ORT Braude College of Engineering, Karmiel.

Figure 1 – Project Illustration



Abstract

This project presents the design and implementation of an autonomous robotic system for moving-target detection using a camera, target tracking, local alerting, and wireless reporting. The motivation for the project stems from the growing need for real-time detection of unauthorized aerial or mobile objects entering restricted areas. In many real-world scenarios, delayed identification of such threats can result in economic damage, security risks, and danger to human life.

The developed system is based on a Raspberry Pi 4 and a Raspberry Pi Camera Module. It scans a defined area, detects a target according to color-based image processing, tracks the target using two servo motors, and sends an alert message to a mobile phone via Wi-Fi and Telegram. Additional local feedback is provided through an LCD display and voice output. A high-intensity LED is used in the prototype as a simulated neutralization or deterrence indicator; it is intended solely for demonstration purposes and does not perform real RF interference.

Image processing was implemented in Python using OpenCV. The system was tested in training and tracking modes, and the results demonstrated successful identification of the target center, conversion of pixel displacement to physical distance, and stable real-time tracking. Engineering challenges such as servo vibration, message-sending latency, and changing illumination conditions were addressed through thresholding, threaded execution, and tuning of image-processing parameters.

Table of Contents

Abstract 2
Abbreviations 4
1. Introduction 5
2. System Description 5
2.1 Functional Definition 5
2.2 Functional Specification 5
2.3 Technical Specification 6
2.3.1 System Modules Table 6
2.3.2 Flow of Operation 9
2.3.3 Principle of Operation 10
3. Engineering Tasks 12
3.1 Requirements-Level Engineering Tasks 12
3.2 Work Performed by the Students 12
3.2.1 Design Stages 12
3.2.2 Preliminary Design 13
3.2.3 Electronic Schematic 15
3.2.4 Simulation and Tests 16
3.3 Engineering Problems and Solutions 19
4. Summary and Discussion 20
4.1 Summary and Conclusions 20
4.2 Decision-Making Process Throughout the Project 20
4.3 Contribution of Each Student 21
4.4 Requirements Compliance Table 21
5. Appendices 22
6. References 22

List of Figures

Figure 1 – Project Illustration 2
Figure 2 – Block Diagram 7
Figure 3 – Flowchart 8
Figure 4 – Simplified Flowchart Describing the System Operation 9
Figure 5 – Raspberry Pi 4 13
Figure 6 – Block Diagram 13
Figure 7 – Electronic Schematic 14

List of Tables

Table 1 – System Components 6
Table 2 – Student Contribution 19
Table 3 – Requirements Compliance Table 20

Abbreviations

Abbreviation	Meaning
GPIO	General Purpose Input/Output
PWM	Pulse Width Modulation
CSI	Camera Serial Interface
HSV	Hue, Saturation, Value
API	Application Programming Interface

1. Introduction

In recent years, the use of drones and other remotely operated mobile platforms has expanded significantly in fields such as industry, security, agriculture, and recreation. Alongside the many benefits these platforms provide, they may also create safety and security challenges when they enter restricted or sensitive areas without authorization.

This project focuses on the development of an autonomous system for color-based detection and tracking of unauthorized moving targets. The prototype is built around a mobile robotic platform equipped with a camera, servo-actuated aiming, local audio and visual alerts, and wireless reporting to a user via Wi-Fi. The system is designed to operate both in daylight and in low-light conditions inside a defined area.

The need for such a system arises from the lack of immediate, low-cost, real-time solutions for detecting and reporting suspicious motion in protected environments. By combining image processing, motion control, and wireless messaging, the project demonstrates a practical academic prototype for real-time monitoring and rapid alerting.

2. System Description

2.1 Functional Definition

A system capable of detecting a moving target by means of a camera, issuing wireless notifications, tracking the target, and simulating local deterrence within the prototype environment.

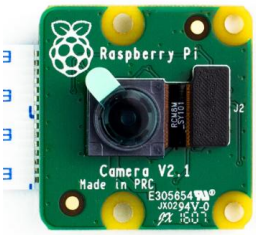
2.2 Functional Specification

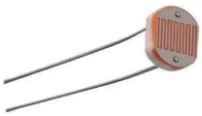
The purpose of the project is to detect targets within a defined open area and generate an alert in order to prevent unauthorized intrusion. The robot autonomously aims toward the target and sends a wireless notification when a threat is identified. From a distance of approximately 5-20 meters, the camera can detect a target of about 30 cm or larger. The Raspberry Pi Camera V2 has a horizontal field of view of 62.2 degrees and a vertical field of view of 48.8 degrees. A high-intensity LED lamp in the range of 3000-5000 lumens or higher is used for prototype illumination and deterrence indication.

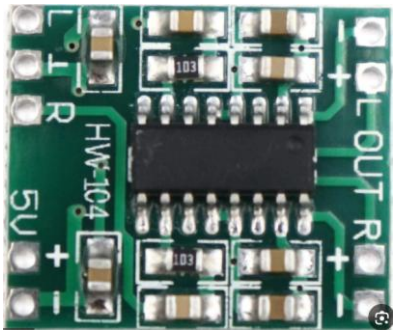
- The system is based on a Raspberry Pi 4 Model B.
- The system includes a Raspberry Pi Camera V2, and the controller processes the video data captured by the camera.
- The system detects moving targets by means of camera-based image processing.
- The camera is moved on two axes using servo motors.
- Image processing is implemented in Python using OpenCV algorithms.
- The user interface is written in Python 3.
- The system sends a notification to the user when a target is detected.
- The system displays the detected target on screen in real time.

2.3 Technical Specification

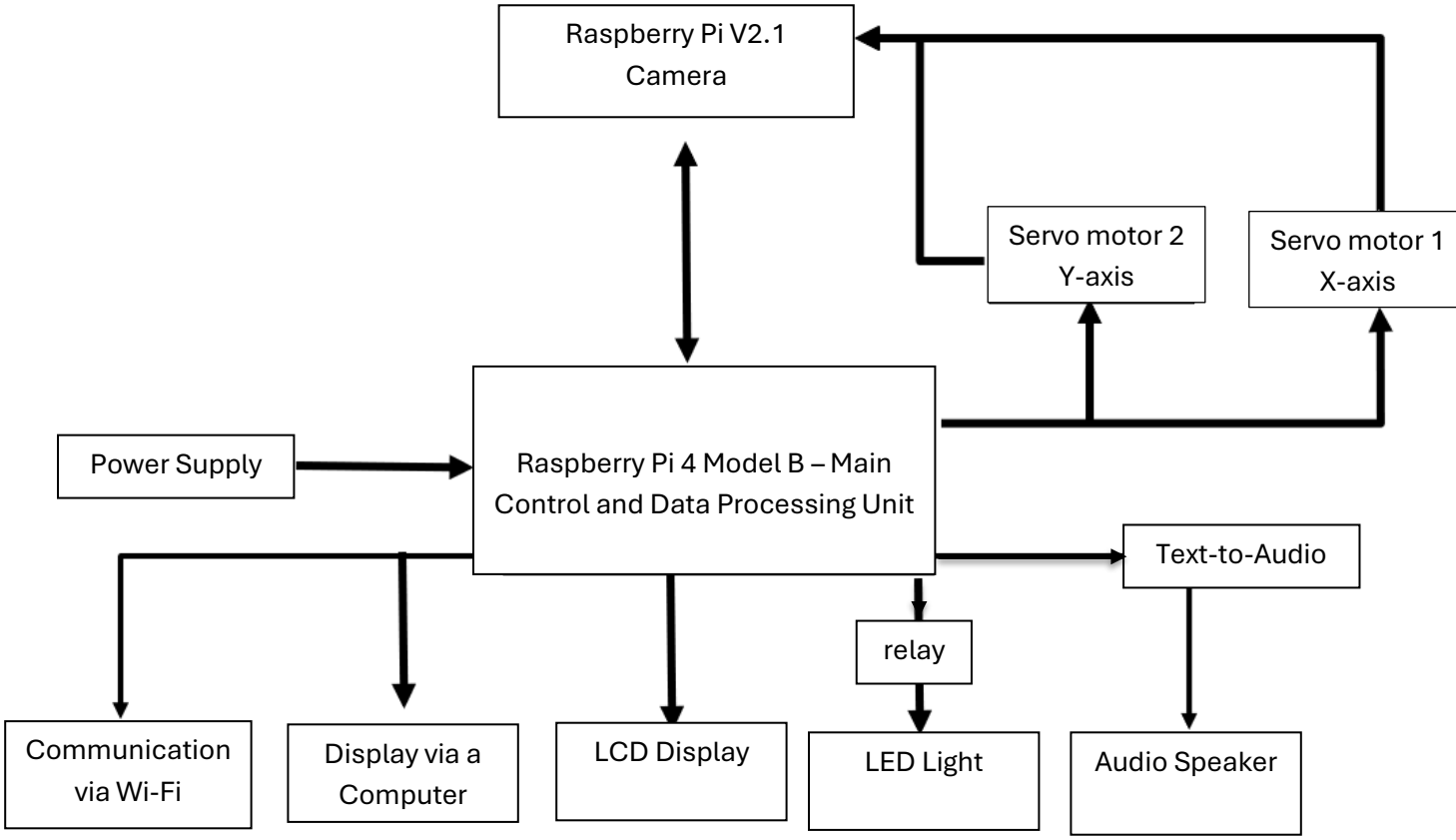
2.3.1 System Modules Table

System Module	Role	Parameters
Raspberry Pi 4 Model B	Main processing and control unit. Processes camera data, displays results, controls robot movement, and enables wireless communication.	Broadcom BCM2711; quad-core Cortex-A72 64-bit SoC at 1.5 GHz; 4 GB LPDDR4 RAM; 5 V DC; min. 3 A
Raspberry Pi Camera V2.1 	Video acquisition for target detection.	Sony IMX219; 25 x 23 x 9 mm; 3 g; 3280 x 2464 pixels, 8 MP
Power Supply	Provides sufficient voltage and current for Raspberry Pi operation.	5 V, 3 A
Wi-Fi Module	Used to send alert notifications.	2.4 GHz and 5.0 GHz

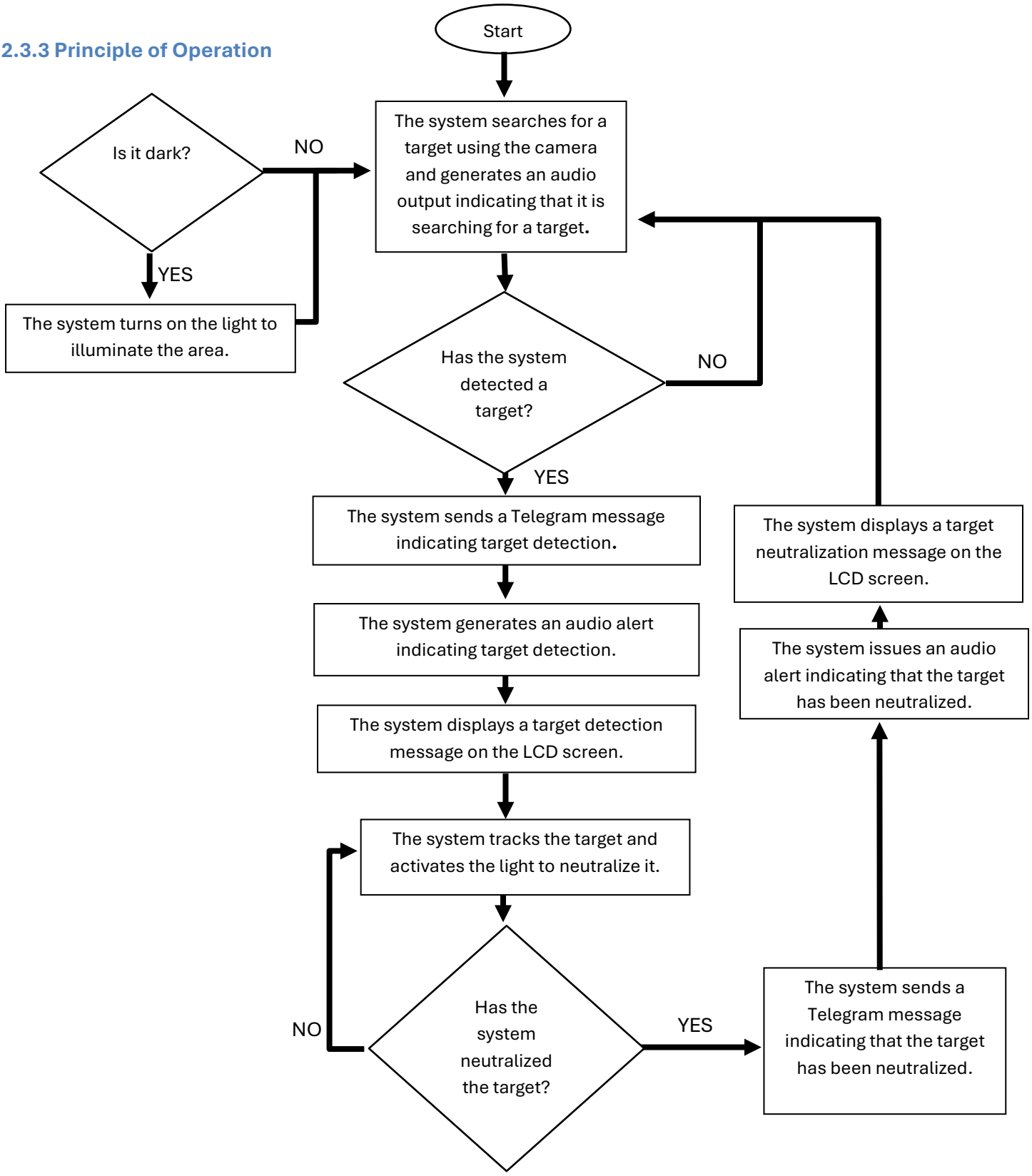
2 x LED Lamps	One lamp simulates target deterrence indication; the second is used for area illumination in darkness.	3.7 V; 1-2.5 A
Audio Speaker	Audio output when a target is detected.	8 ohm; 10 W
ADC ADS7830	Analog-to-digital converter.	2.5-5 V
2 x Relay	Switches for opening/closing the LED lamp circuits.	250 V AC / 5 V DC; 10 A
Photoresistor 	Converts light level into voltage variation.	5-200 kOhm
2 x 18650 Lithium Batteries	Power source.	3.7 V; up to 2.5 A

HW-104 	Text-to-speech module.	5 V DC; 46-220 mA
2 x SG90 Servo Motors 	Move the camera and lamp to scan the area, track the target, and point toward it.	Approx. 180 degrees; 0.1 s/60 degrees; 2.5 kg-cm; 4.8-6 V
LCD1602 I2C 	Display output for target detection and neutralization status.	5 V; 80 x 36 mm

2.3.2 Block Diagram

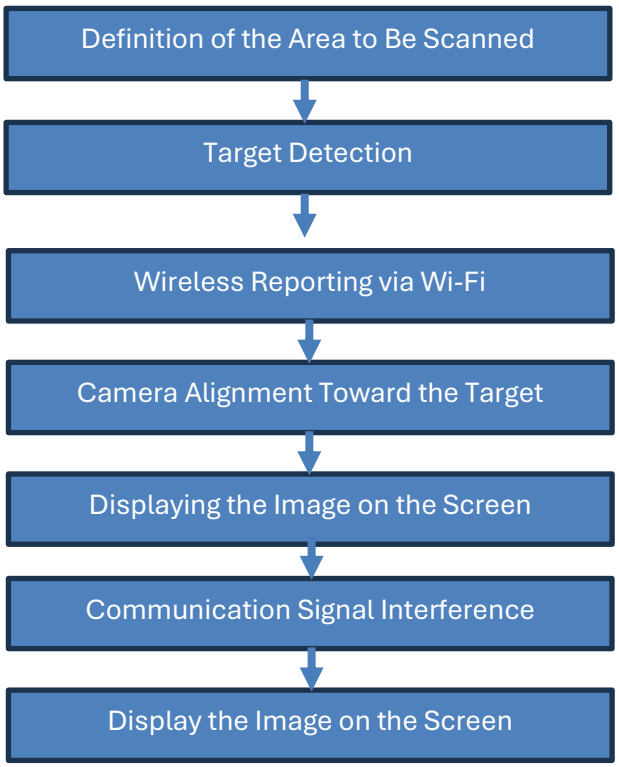


2.3.3 Principle of Operation



2.3.4 Principle of Operation

1. **Target Detection:** The system begins by scanning a predefined area using the Raspberry Pi Camera V2. Captured frames are processed in Python with OpenCV in order to identify a defined target, primarily on the basis of color.
2. **Wireless Reporting:** Once a target is detected, the system sends a wireless notification to the user through Wi-Fi and Telegram. This enables real-time awareness of a potential threat inside the monitored area.
3. **Camera Alignment Toward the Target:** After detection, the system uses two servo motors to align the camera toward the target. The target position is calculated relative to the frame center, and the camera is moved accordingly so that the system can continue tracking the object across the horizontal and vertical axes.
4. **Image Display:** The image of the target is displayed in real time for monitoring purposes. In the prototype, a high-intensity LED is activated as a visual indicator representing simulated deterrence or neutralization. This is a demonstration-only mechanism and should not be interpreted as a real jamming or weaponized function.
5. **Autonomous Operation:** The robot operates autonomously and performs detection, alignment, display, alerting, and simulated deterrence without requiring continuous human intervention.



3. Engineering Tasks

3.1 Requirements-Level Engineering Tasks

- Writing the project proposal.
- Planning the system modules.
- Learning the Raspberry Pi operating environment.
- Studying the OpenCV library.
- Learning the operating principles of servo motors.
- Designing and implementing the motion algorithm.
- Selecting suitable components for the project.
- Implementing wireless communication between the robot controller and the central computer/mobile device.
- Integrating the system modules.
- Testing the system and evaluating the results.
- Writing the final project book.

3.2 Work Performed by the Students

3.2.1 Design Stages

1. **<u>System characterization stage:</u>**
   - **Definition of the project goals:**
     - Understanding the need, system objectives, and desired outcome.
     - Definition of functional and non-functional requirements such as performance, robustness, and response time.
   - **Risk analysis:**
     - including target-detection errors, wireless communication failures, and robotic faults.
2. **<u>Hardware design stage:</u>**
   - **Selection of core hardware components:**
     - Raspberry Pi 4, NoIR / camera module, servo motors, and LED lamp.
     - Selection of power components, cabling, and adapters.
   - **Preparation of an electrical schematic for all hardware connections.**
3. **<u>Software design stage:</u>**
   - **Design of the software architecture and module separation:**
     - image processing, servo control, user interface, and wireless reporting.
     - Definition of the camera workflow and the OpenCV-based image-processing algorithm for target detection.
     - Planning of the Python 3 user interface.
     - Development planning for Wi-Fi reporting and alerts.
     - Integration planning for servo motors and LED control using Python.
     - Design of target-detection and target-tracking algorithms.
4. **<u>Development stage:</u>**

- **Assembly of the hardware components and initial component-by-component testing.**
- **Development of the initial software, including camera connection, image processing, and motor control.**
- **Development of the user interface and the wireless reporting code, including image saving.**

5. **<u>Integration stage:</u>**
   - **Combining the hardware and software subsystems:**
     - Testing communication among the camera, motors, lamps, and reporting modules.
     - Verifying full system compatibility.
6. **<u>Testing stage:</u>**
   - Functional tests
     - target detection at the required size and range.
     - Accuracy tests for target recognition and camera movement response.
7. **<u>Correction and improvement stage:</u>**
   - Troubleshooting the faults discovered during testing.
   - Improving the detection algorithm for better accuracy.
   - Optimizing the system response time.
8. **<u>Field deployment and experimentation stage:</u>**
   - Operating the robot in a defined test area.
   - Monitoring system successes and failures under real conditions.
   - Applying post-trial improvements based on field-test analysis.

3.2.2 Preliminary Design

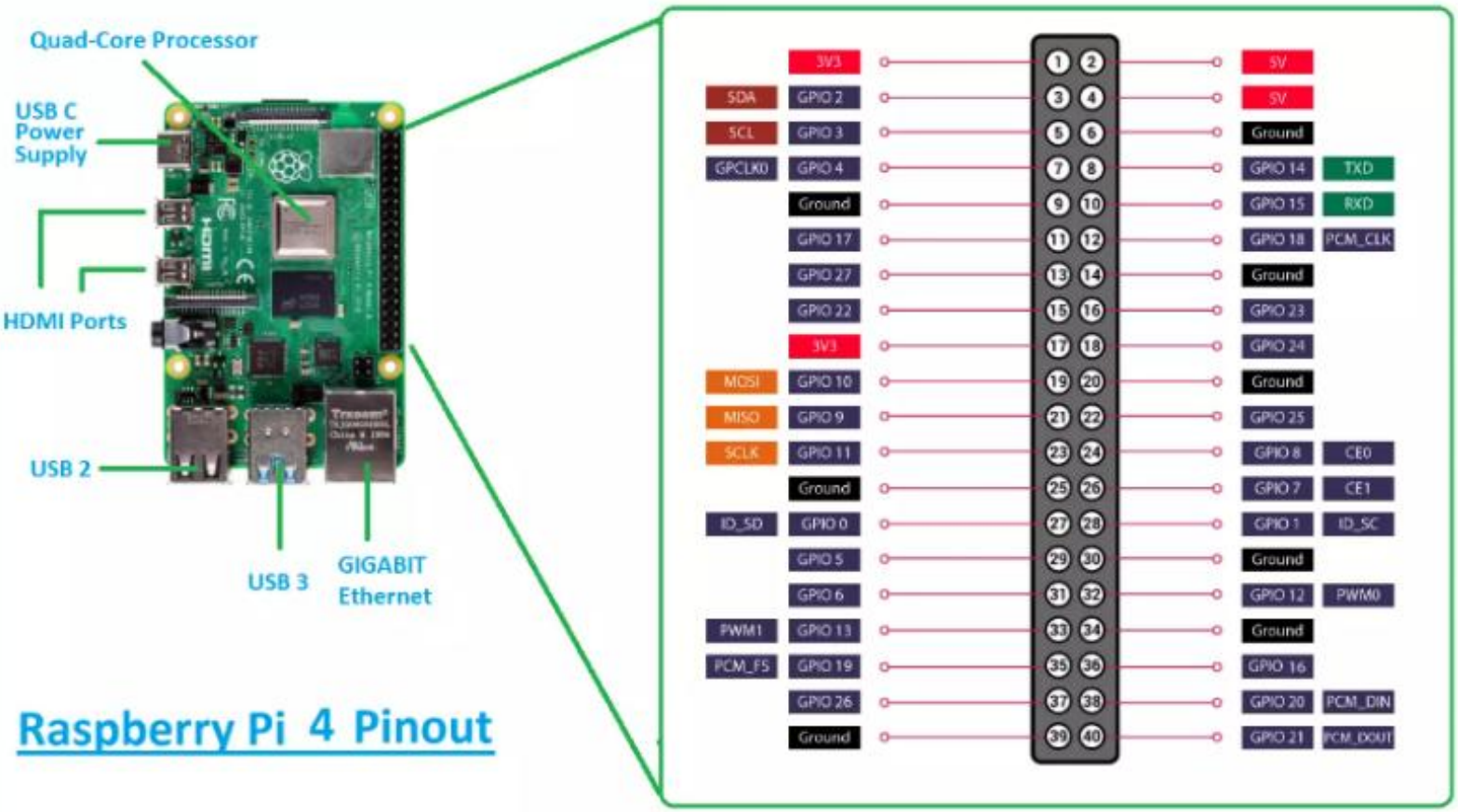
A Python-based development environment running on Linux was selected for the Raspberry Pi. This choice was driven by the team's familiarity with the language and platform, Python's ease of use, and the availability of mature libraries for image processing, GPIO control, networking, and hardware integration.

Python provides access to camera-oriented tools and to OpenCV for real-time image processing. These libraries enabled convenient implementation of object tracking, target recognition, image analysis, and interaction with the camera module.

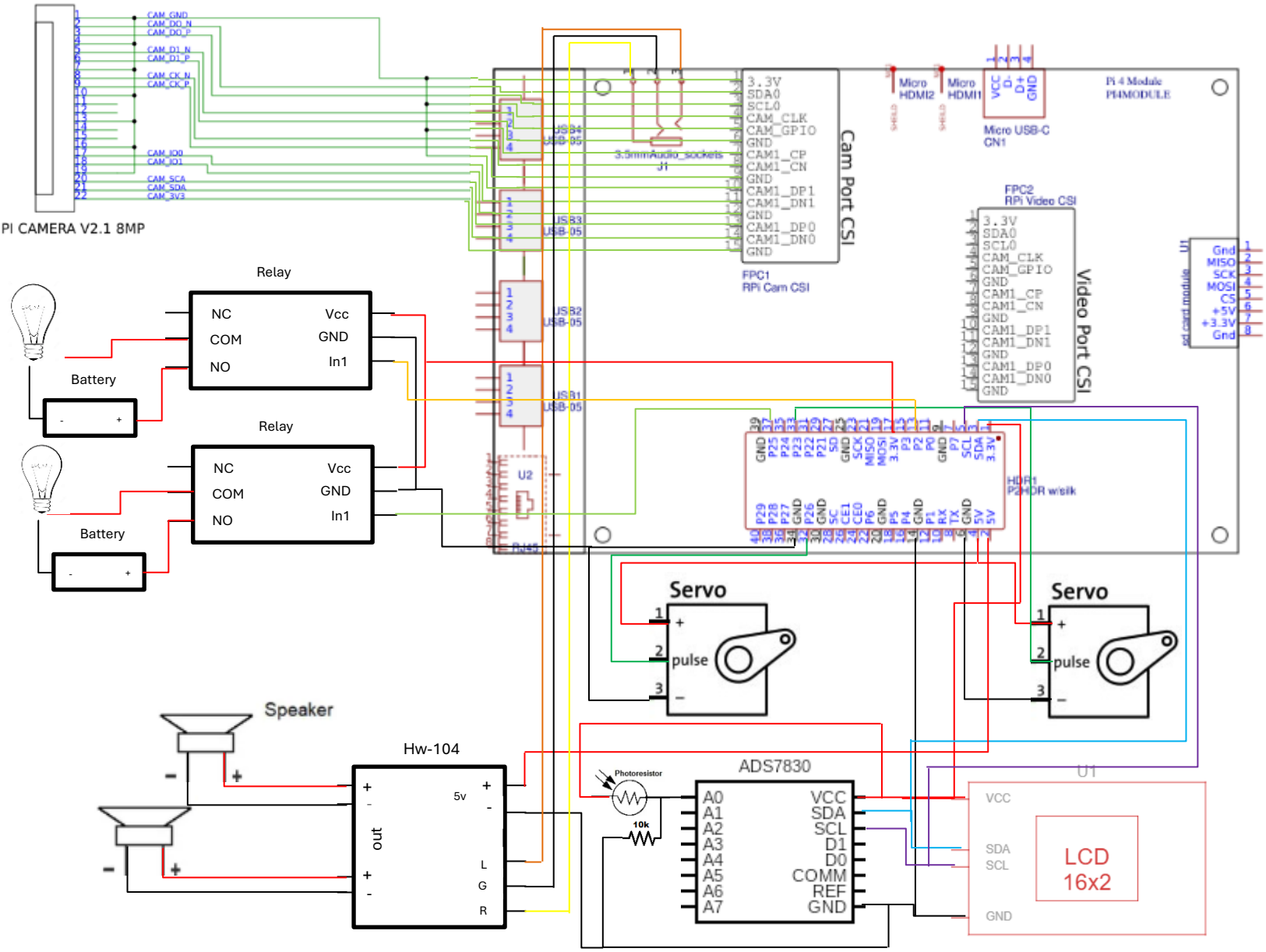
Python also offers flexibility through a broad ecosystem of libraries for wireless communication and GPIO control. This made it possible to control servo motors, send notifications through Wi-Fi, and maintain modular, readable code.

The Raspberry Pi 4 Model B was selected as the main controller because of its relatively strong processing power, built-in networking capabilities, and support for multiple interfaces such as GPIO, USB 3.0, HDMI, and CSI camera connectivity. These characteristics made it suitable for real-time image processing and for handling several peripheral devices simultaneously.

A Telegram-based alert module was used to send notifications whenever the target was found. Once the algorithm identifies the object, a dedicated Telegram thread or asynchronous process sends a real-time alert through the Telegram API.


Quad-Core Processor
USB C Power Supply
HDMI Ports
USB 2
USB 3
GIGABIT Ethernet
Raspberry Pi 4 Pinout
3V3
SDA
GPIO 2
SCL
GPIO 3
GPCLK0
GPIO 4
Ground
GPIO 17
GPIO 27
GPIO 22
3V3
MOSI
GPIO 10
MISO
GPIO 9
SCLK
GPIO 11
Ground
ID_SD
GPIO 0
GPIO 5
GPIO 6
PWM1
GPIO 13
PCM_FS
GPIO 19
GPIO 26
Ground
1 2
3 4
5 6
7 8
9 10
11 12
13 14
15 16
17 18
19 20
21 22
23 24
25 26
27 28
29 30
31 32
33 34
35 36
37 38
39 40
5V
5V
Ground
GPIO 14
TXD
GPIO 15
RXD
GPIO 18
PCM_CLK
Ground
GPIO 23
GPIO 24
Ground
GPIO 25
GPIO 8
CE0
GPIO 7
CE1
GPIO 1
ID_SC
Ground
GPIO 12
PWM0
Ground
GPIO 16
GPIO 20
PCM_DIN
GPIO 21
PCM_DOUT

3.2.3 Electronic Schematic



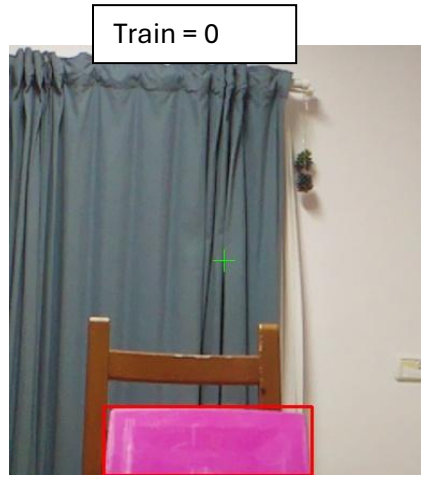
3.2.4 Simulation and Tests

At the initial testing stage, a colored target object was selected. In our case, a pink object approximately 30 cm wide and more than 35 cm long was used as the moving target to be detected and tracked.

The system includes two modes of operation:

Train = 0 ➔ In Train mode, the machine is taught which color it should follow.

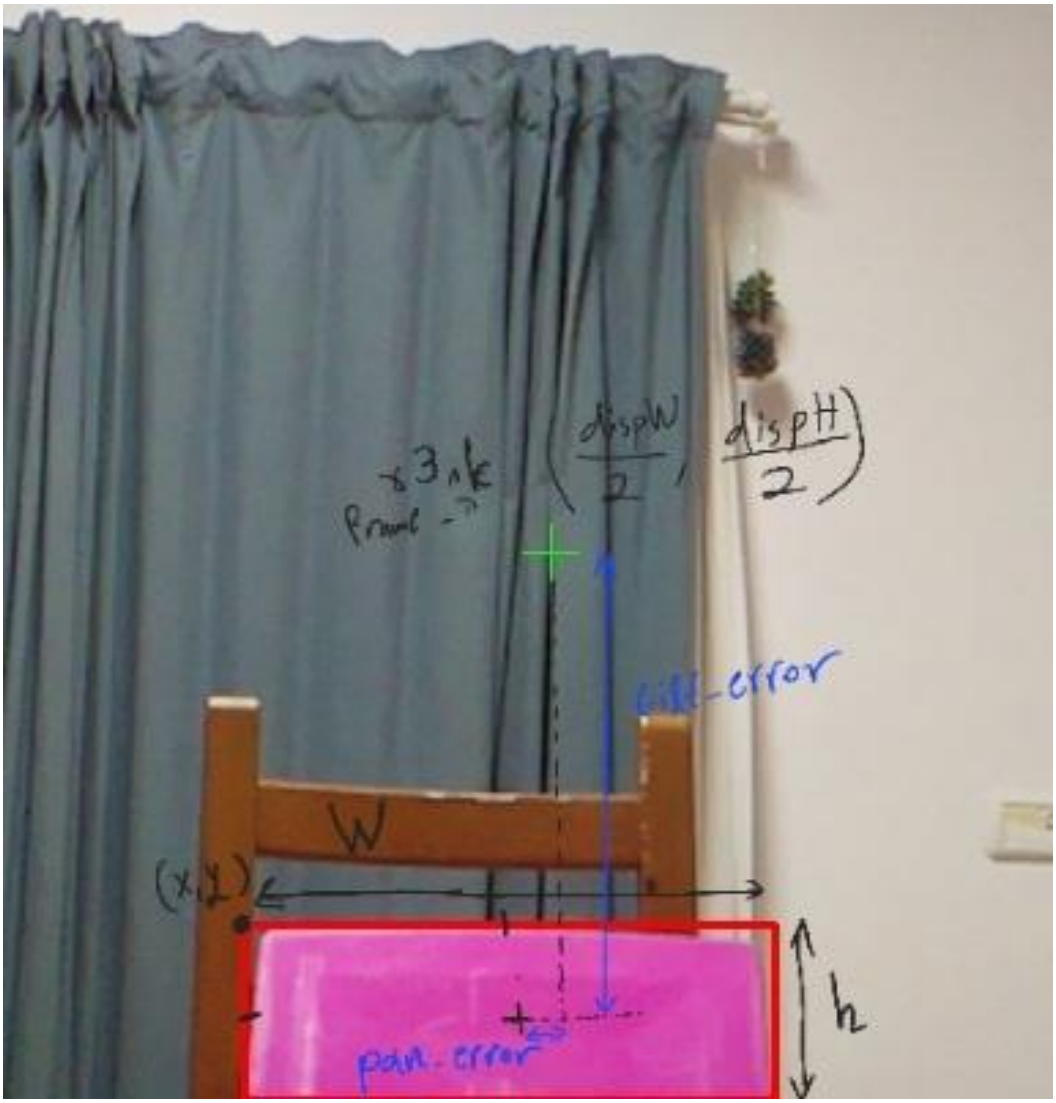
Track = 1 ➔ In Track mode, the system enters automatic operation and follows the center of the selected target in real time according to the color learned during Train mode.



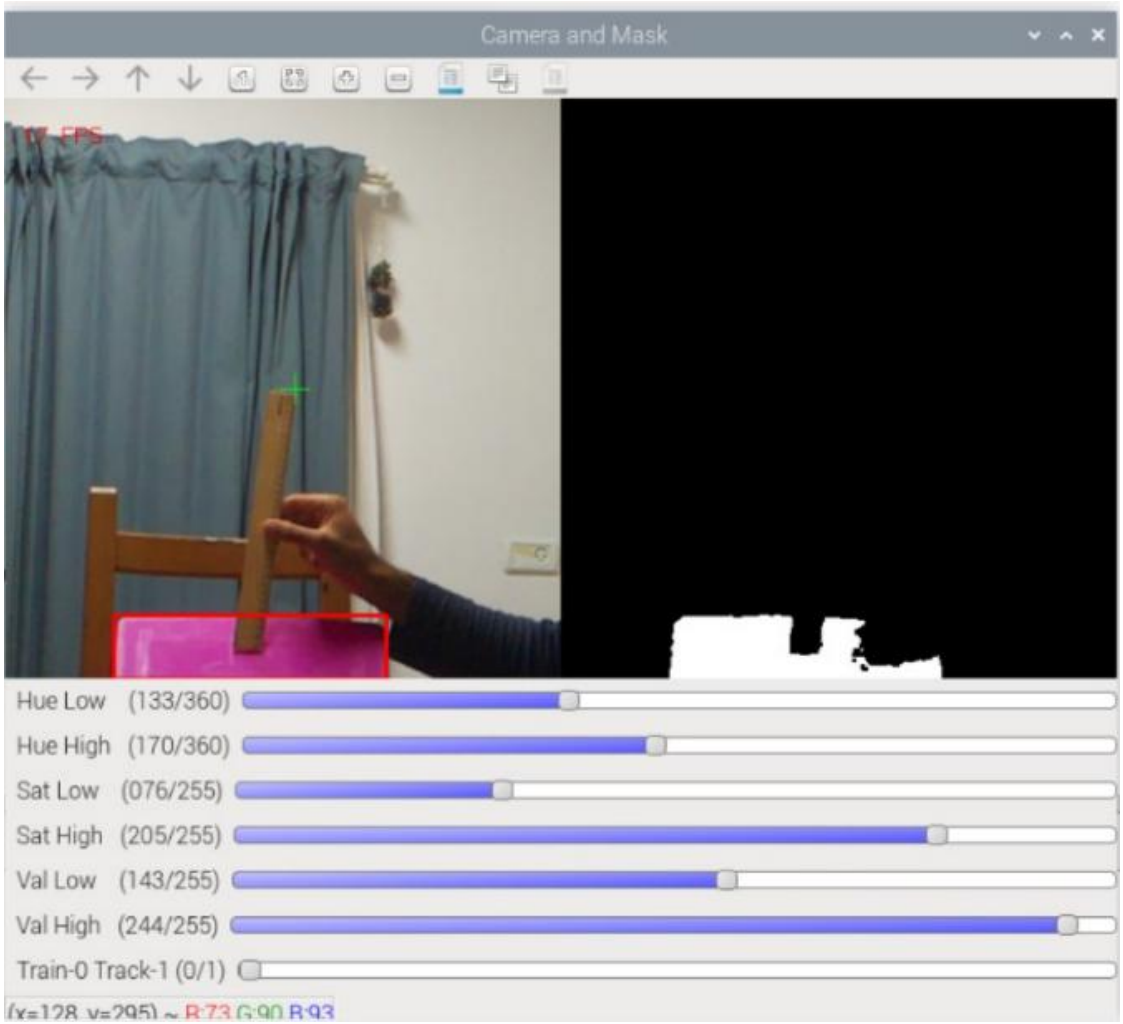
When contours are found in the frame, the algorithm selects the contour with the largest area, extracts the x and y coordinates as well as the width and height of the bounding rectangle, draws a rectangle around the detected object, and calculates the horizontal and vertical errors relative to the frame center.

$$pan_\text{error} = (x + w/2) - dispW/2$$

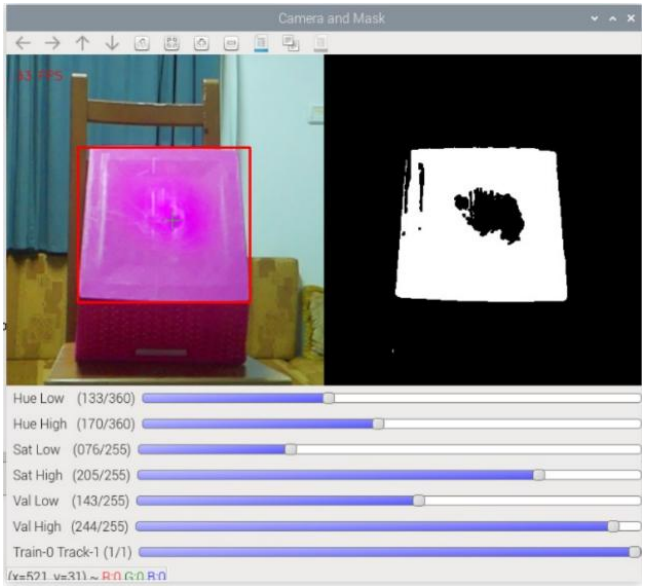
$$\text{tilt_error} = (y + h/2) - dispH/2$$



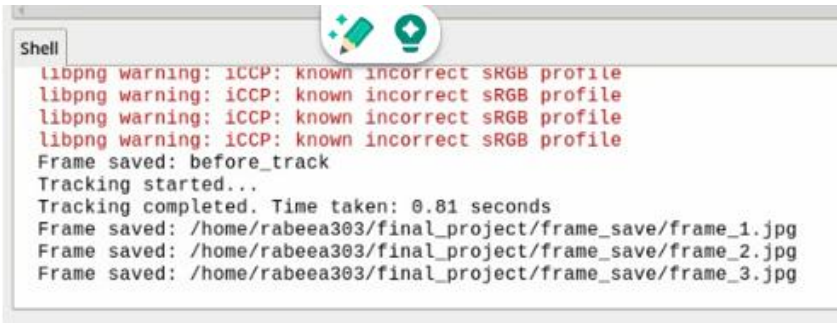
To validate the conversion from pixels to centimeters, the camera was placed in an initial position of (0, 0) and the target was located at a distance of 1.5 m from the camera. A ruler-based reference was used so that the center of the target was approximately 30 cm away from the frame center. The shell output showed the distance from the center of the target in both X and Y directions in centimeters. Using the Pythagorean theorem, the calculated distance matched the manually measured distance with good approximation.



In Track mode, the system successfully followed the target and pointed toward its center with good accuracy. The crosshair displayed in the tracking view remained close to the true center of the target, and the distance from the target center approached zero during steady tracking.



The measured time required for the camera to reach the center of the target while saving images during tracking was approximately 0.81 seconds.



3.3 Engineering Problems and Solutions

Problem 1: Improving target-tracking speed and accuracy

- Challenge: During development, the tracking performance was not accurate enough and the response was sometimes slow, which caused unstable motor movement and missed fast-moving targets.
- Solution: A motion-suppression threshold was defined so that the motors would move only when the pan and tilt errors exceeded 15 pixels. In addition, a more accurate pixel-to-angle conversion was used to update the servo angles according to the relative target position.

Problem 2: Camera freezing while sending Telegram messages and saving images

- Challenge: When the camera detected the target and the system tried to send a Telegram message or save an image, the program could freeze and disrupt the tracking loop.
- Solution: A separate thread was introduced for Telegram messaging and image saving. This enabled the camera and the main image-processing loop to continue running without blocking, and the problem was resolved successfully.

Problem 3: Tracking accuracy under changing illumination

- Challenge: Under weak lighting or strong illumination, the quality of target recognition decreased. Noise and loss of focus reduced the system's ability to track the target accurately.
- Solution: To address this issue, the image-processing filters were tuned, including saturation and brightness thresholds, in order to reduce the effect of lighting conditions on detection quality.

4. Summary and Discussion

4.1 Summary and Conclusions

This project resulted in the development of a robotic prototype for moving-target detection, tracking, local alerting, and wireless reporting. The system was motivated by the need to identify suspicious objects entering restricted spaces and to provide rapid situational awareness. Using a Raspberry Pi 4, a camera, and servo motors, the system detects color-based targets in an open area, follows them, and activates a simulated neutralization indicator represented by a high-intensity LED. An automatic warning is also sent through Wi-Fi to the user.

The image-processing flow was implemented in Python with OpenCV and included real-time frame analysis, target localization, and autonomous tracking. The project required the integration of hardware, software, control logic, and communication modules.

From an educational perspective, the project was both challenging and rewarding. It required the application of knowledge acquired during the degree, the solution of integration problems between the camera and the motors, and the management of asynchronous communication tasks. The successful operation of the system provided a meaningful demonstration of real-time embedded vision in an academic prototype.

The project also created opportunities for future improvement, such as higher detection accuracy, improved robustness to lighting changes, and faster response time. Overall, it contributed significantly to the students' professional and personal development.

4.2 Decision-Making Process Throughout the Project

During development, several technical challenges required critical decision-making to ensure proper system operation.

At the initial design stage, the Raspberry Pi 4 was chosen as the central processing unit due to its image-processing capabilities and built-in wireless connectivity. The Raspberry Pi Camera V2 was selected because of its resolution and suitability for object detection.

At one point, a hardware fault prevented the camera from operating. After systematic testing, the connection cable was checked and found to be defective. Replacing the cable resolved the issue.

The SG90 servo motors were selected for camera control because they offer light weight, acceptable torque, and approximately 180 degrees of motion, which matched the needs of the system.

4.3 Contribution of Each Student

Rabea	Aslan
Responsible for hardware integration: Raspberry Pi, camera, servo motors, and LCD.	Responsible for hardware assembly: Raspberry Pi, camera, and servo motors.
Designed the motion and servo-control algorithm.	Developed the color-based object detection code.
Documented the development stages.	Integrated the system with the wireless communication interface.
Wrote code for tracking the target.	Wrote code for tracking the target.
Contributed to writing the final project report.	Contributed to writing the final project report.
Prepared the project presentation.	Prepared the project presentation.

4.4 Requirements Compliance Table

Requirement	Status
Writing the project proposal	Completed
Considering design constraints and planning factors	Completed
Using servo motors on two axes	Completed
Using wireless Wi-Fi communication	Completed
Understanding the use of the OpenCV library in Python	Completed
Implementing image-processing code for tracking a moving target	Completed
Integrating the system modules	Completed
Testing the system, locating faults, and correcting them	Completed
Writing the project book	Completed

5. Appendices

Appendix A - Code files

Appendix B - Servo motor

Appendix C - Raspberry Pi Camera Module

Appendix D - ADS7830

Appendix E - LCD

Appendix F - HW-104

Appendix G - Relay

Appendix H - System algorithms

Appendix I - Telegram bot creation guide

6. References

1) Raspberry Pi 4 model B datasheet
raspberry-pi-4-product-brief.pdf (raspberrypi.com)

2) Raspberry Pi V2 Camera Module datasheet
https://www.raspberrypi.com/documentation/accessories/camera.html

3) Servo motor datasheet
https://www.friendlywire.com/projects/ne555-servo-safe/SG90-datasheet.pdf

4) OpenCV-Python
https://codedamn.com/news/python/beginners-guide-to-opencv-in-python

5) Free Raspberry Pi tutorials for beginners
https://www.youtube.com/watch?v=1WDagiA8fdU&list=PLGs0VKk2DiYxdMjCJmcP6jt4Yw6OHK85O

6) Multirotor Wikipedia
https://en.wikipedia.org/wiki/Multirotor